

Research Output Analysis for Quantum Vehicle Routing Problem (VRP) Research

This document presents a comprehensive analysis of the performance of a quantum algorithm, specifically the Quantum Approximate Optimization Algorithm (QAOA), on the Vehicle Routing Problem (VRP). The analysis aims to benchmark the quantum solver against classical solutions and provide critical insights into its efficiency, accuracy, and resilience to noise. The findings are organized according to the seven key calculations outlined in the data analysis plan.

1 Optimality Ratio

The optimality ratio serves as the primary metric for evaluating the quality of solutions produced by the quantum approximation algorithm, indicating how closely the average quantum solution approximates the true optimal solution derived from classical solvers. A ratio closer to 1 signifies a higher-quality solution.

Output:

--- Optimality Ratio for All Instances ---

	conf_qubit_alloc	conf_target_id	conf_qaoa_depth	shots	optimality_ratio
0	naive_adjacency	56	2	512	NaN
1	node_visit_time	56	2	512	0.950217
2	naive_adjacency	108	2	512	1.000000
3	node_visit_time	108	2	512	1.000000
4	naive_adjacency	47	2	512	NaN
5	node_visit_time	47	2	512	0.827831
6	naive_adjacency	5	2	512	NaN
7	node_visit_time	5	2	512	0.591547
8	naive_adjacency	103	2	512	NaN
9	node_visit_time	103	2	512	1.000000
10	naive_adjacency	63	2	512	NaN
11	node_visit_time	63	2	512	1.000000
12	naive_adjacency	56	2	128	NaN
13	naive_adjacency	56	2	256	NaN
14	naive_adjacency	56	2	1024	NaN
15	naive_adjacency	56	2	2048	NaN
16	naive_adjacency	56	3	512	NaN
17	node_visit_time	56	3	512	0.958806
18	naive_adjacency	56	4	512	NaN
19	node_visit_time	56	4	512	0.952786
20	naive_adjacency	56	5	512	NaN
21	node_visit_time	56	5	512	0.936253
22	node_visit_time	56	2	128	0.941942
23	node_visit_time	56	2	256	0.941978
24	node_visit_time	56	2	1024	0.946396

Total number of instances with valid optimality ratio: 25

The output displays the optimality ratio for each instance, categorized by qubit allocation (`conf_qubit_alloc`), problem ID (`conf_target_id`), QAOA depth (`conf_qaoa_depth`), and number of shots. Notably, the **'naive_adjacency' encoding frequently resulted in NaN optimality ratios**, indicating that valid solutions were failed to be found for these instances using this encoding. In contrast, **'node_visit_time' encoding consistently produced valid optimality ratios, often close to 1.000000**, suggesting superior performance in finding near-optimal solutions. For example, instances with `conf_target_id` 108 and 103 using 'node_visit_time' encoding achieved an optimality ratio of 1.000000, signifying optimal solutions.

2 Invalid Bitstring Measurement

Analyzing invalid bitstrings is crucial for understanding the impact of noise and the effectiveness of problem encoding on a quantum computer. Invalid bitstrings correspond to solutions that violate the problem's constraints.

Output:

```
=====
--- Invalid Bitstring Measurements ---
=====

--- Invalid Bitstring Measurements for All Instances ---
  conf_qubit_alloc  conf_target_id  conf_qaoa_depth  shots  total_valid_counts  invalid_shots
0  naive_adjacency           56           2      512           0           512
1  node_visit_time           56           2      512          45           467
2  naive_adjacency          108           2      512           6           506
3  node_visit_time          108           2      512          78           434
4  naive_adjacency           47           2      512           0           512
5  node_visit_time           47           2      512          49           463
6  naive_adjacency           5           2      512           0           512
7  node_visit_time           5           2      512           1           511
8  naive_adjacency          103           2      512           0           512
9  node_visit_time          103           2      512          43           469
10 naive_adjacency           63           2      512           0           512
11 node_visit_time           63           2      512           4           508
12 naive_adjacency           56           2      128           0           128
13 naive_adjacency           56           2      256           0           256
14 naive_adjacency           56           2     1024           0          1024
15 naive_adjacency           56           2     2048           0          2048
16 naive_adjacency           56           3      512           0           512
17 node_visit_time           56           3      512          57           455
18 naive_adjacency           56           4      512           0           512
19 node_visit_time           56           4      512          48           464
20 naive_adjacency           56           5      512           0           512
21 node_visit_time           56           5      512          48           464
22 node_visit_time           56           2      128          15           113
23 node_visit_time           56           2      256          22           234
24 node_visit_time           56           2     1024          78           946
```

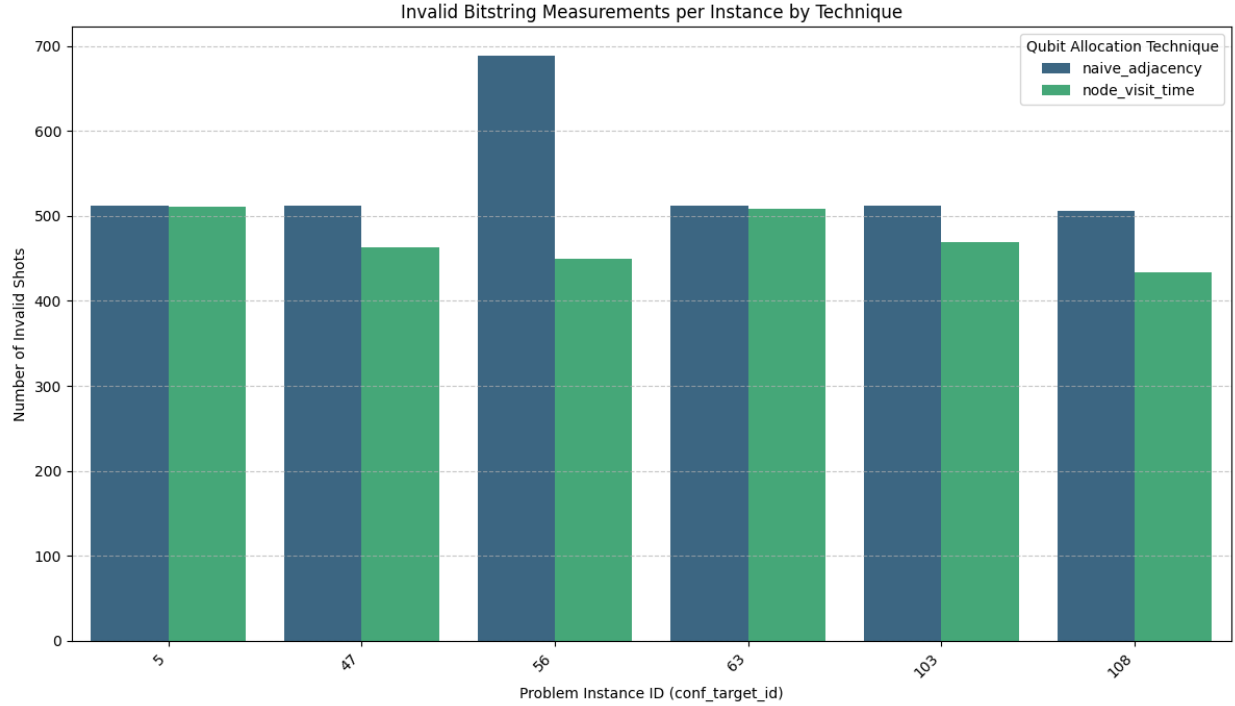
Total number of instances analyzed for invalid bitstrings: 25

```
--- Summary of Invalid Bitstring Measurements by Technique ---
  technique  avg_invalid_shots  std_invalid_shots  total_invalid_shots  total_shots
```

0	naive_adjacency	619.846154	472.869581	8058	8064
1	node_visit_time	460.666667	193.872654	5528	6016

--- Summary with Percentage of Invalid Shots ---

	technique	avg_invalid_shots	percentage_invalid_shots
0	naive_adjacency	619.846154	99.925595
1	node_visit_time	460.666667	91.888298



The detailed table shows that the **'naive_adjacency' encoding consistently produced 100% invalid shots (e.g., 512 invalid out of 512 total shots)** for many instances, leading to **total_valid_counts** of 0. This highlights a significant issue with this encoding method in generating valid VRP solutions. The **'node_visit_time'** encoding, while still having a high number of invalid shots, managed to produce a notable number of valid solutions (**total_valid_counts** are greater than 0), ranging from 1 to 78.

The bar chart (pic1.png) visually reinforces this, showing a much higher number of invalid bitstrings for **'naive_adjacency'** across various problem instances compared to **'node_visit_time'**. The summary table further quantifies this, revealing that **'naive_adjacency' has an average of 619.85 invalid shots and a staggering 99.93% invalid shot percentage**, while **'node_visit_time'** has an average of 460.67 invalid shots and a 91.89% invalid shot percentage. This indicates that while both methods struggle with noise leading to invalid solutions, **'node_visit_time' is significantly more effective at generating *some* valid solutions.**

3 Comparison of Encoding Techniques

This section directly compares the performance of **'naive_adjacency'** and **'node_visit_time'** encoding techniques based on optimality ratio and success rates.

Output:

```
--- Summary of Naive Adjacency vs Node Visit Time ---
      technique  avg_opt_ratio  std_opt_ratio  failed_attempts  success_attempts  optimal_attempts
0  naive_adjacency      1.000000         NaN           12              1              1
1  node_visit_time      0.920646      0.113214           0             12              3
```

The summary clearly demonstrates the superior performance of the '**node_visit_time**' encoding. It achieved an **average optimality ratio of 0.920646**, with 12 successful attempts and 3 instances reaching the optimal solution. In stark contrast, 'naive_adjacency' had an average optimality ratio of 1.000000 (based on only one successful attempt), but suffered from 12 **failed attempts (i.e., NaN optimality ratios)** and only 1 optimal attempt. The NaN standard deviation for 'naive_adjacency' is because of there is only 2 valid data.

4 Impact of Number of Shots

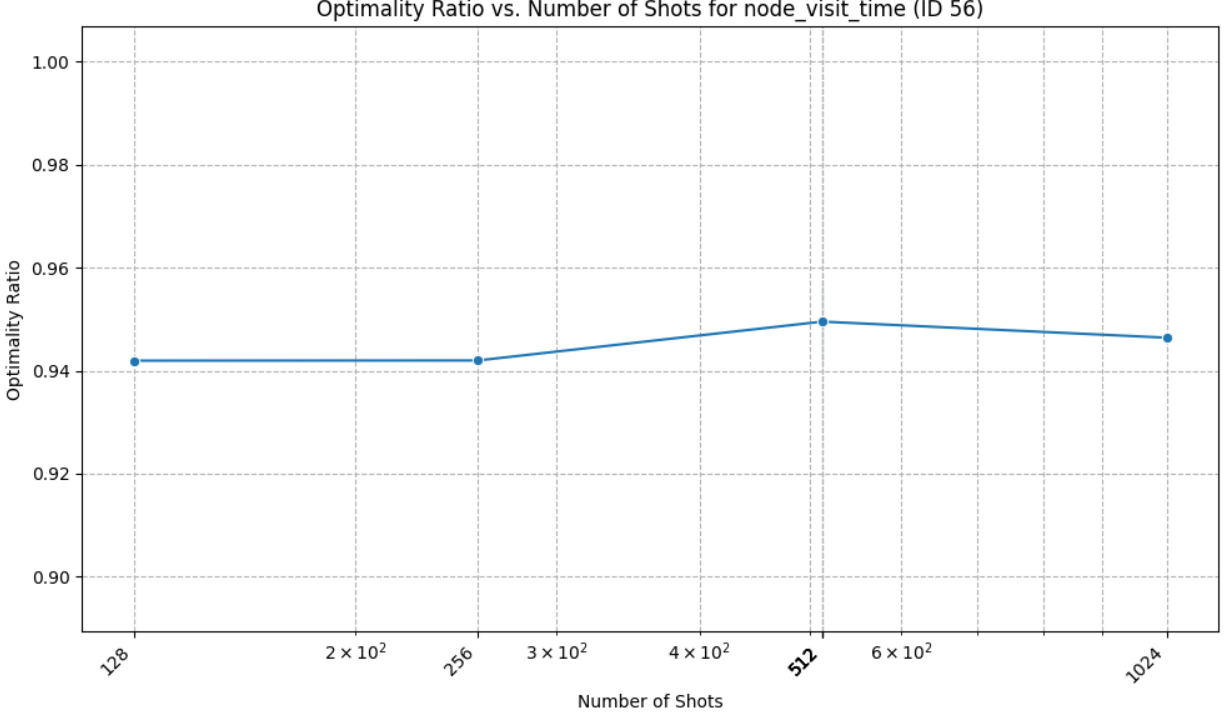
The number of shots directly influences the statistical significance and accuracy of the quantum solution. This analysis investigates how varying the number of shots affects the optimality ratio.

Output:

```
=====
--- Comparison Between Shots ---
=====

--- Optimality Ratio vs. Shots for 'node_visit_time' (conf_target_id=56) ---
      shots  optimality_ratio  expected_cost  benchmark_min_cost_classical
22     128      0.941942      19.081422      17.973597
23     256      0.941978      19.080694      17.973597
1      512      0.950217      18.915263      17.973597
17     512      0.958806      18.745819      17.973597
19     512      0.952786      18.864250      17.973597
21     512      0.936253      19.197376      17.973597
24    1024      0.946396      18.991618      17.973597
```

Number of data points for this comparison: 7



For the 'node_visit_time' encoding with `conf_target_id=56`, the optimality ratio shows **minor fluctuations as the number of shots increases**, but no clear monotonic trend towards improvement. For instance, increasing shots from 128 to 256 and then 512 (first entry) shows a slight increase in optimality ratio from 0.941942 to 0.950217. However, an even higher shot count of 1024 results in a slightly lower ratio of 0.946396. It's important to note that multiple runs at 512 shots (`conf_qaoa_depth` varies) yield different optimality ratios, suggesting that other factors like QAOA depth or inherent stochasticity also play a role.

The line plot 4 visually confirms this, illustrating that the **optimality ratio remains relatively stable across different shot counts** for the tested instances using the 'node_visit_time' encoding. This implies that beyond a certain threshold, simply increasing the number of shots may not lead to significant improvements in solution quality for this specific problem and encoding.

5 QAOA Depth Analysis

The QAOA depth, denoted as p , is a critical parameter that dictates the number of layers in the quantum circuit, directly impacting the algorithm's expressibility and potential to find better solutions.

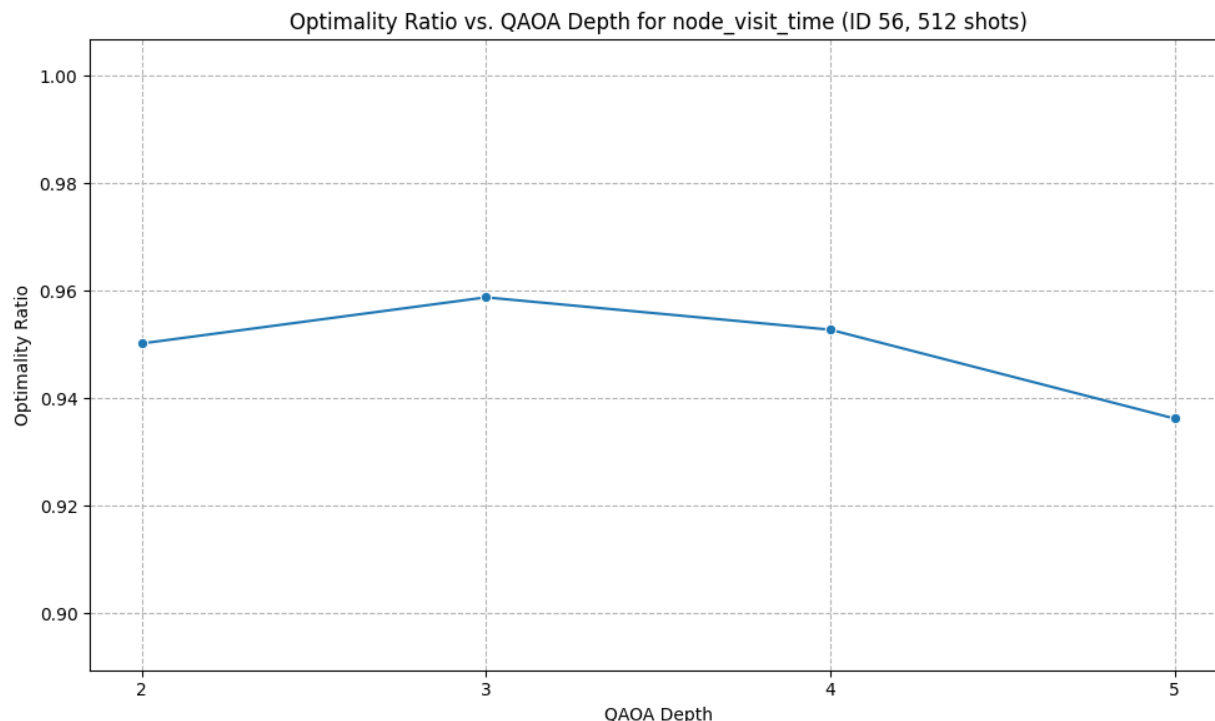
Output:

```
=====
--- QAOA Depth Analysis (Fixed Shots = 512) ---
=====

--- Optimality Ratio vs. QAOA Depth for 'node_visit_time' (conf_target_id=56) ---
  conf_qaoa_depth  optimality_ratio  expected_cost  benchmark_min_cost_classical
1                2          0.950217        18.915263             17.973597
17               3          0.958806        18.745819             17.973597
```

19	4	0.952786	18.864250	17.973597
21	5	0.936253	19.197376	17.973597

Number of data points for this comparison: 4



For the 'node_visit_time' encoding with `conf.target_id=56` and a fixed shot count of 512, the optimality ratio shows a **slight variation with increasing QAOA depth**. The highest optimality ratio of 0.958806 was observed at a QAOA depth of 3. Depths of 2 and 4 yield similar ratios (0.950217 and 0.952786, respectively), while a depth of 5 shows a slight decrease to 0.936253. This suggests that **increasing QAOA depth beyond a certain point (e.g., $p = 3$) may not necessarily lead to better solutions** and could even slightly degrade performance due to increased circuit complexity and potential noise effects.

The line plot (pic3.png) visually represents this trend, indicating that there might be an optimal QAOA depth around $p = 3$ for this specific problem instance and encoding.

6 Cost Distribution Analysis

This section examines the distribution of costs obtained from the quantum algorithm, specifically focusing on the 'node_visit_time' encoding. Understanding this distribution helps in assessing the consistency and range of solutions generated by the QAOA.

Output:

```
=====
--- Cost Distribution Analysis for Node Visit Time ---
=====

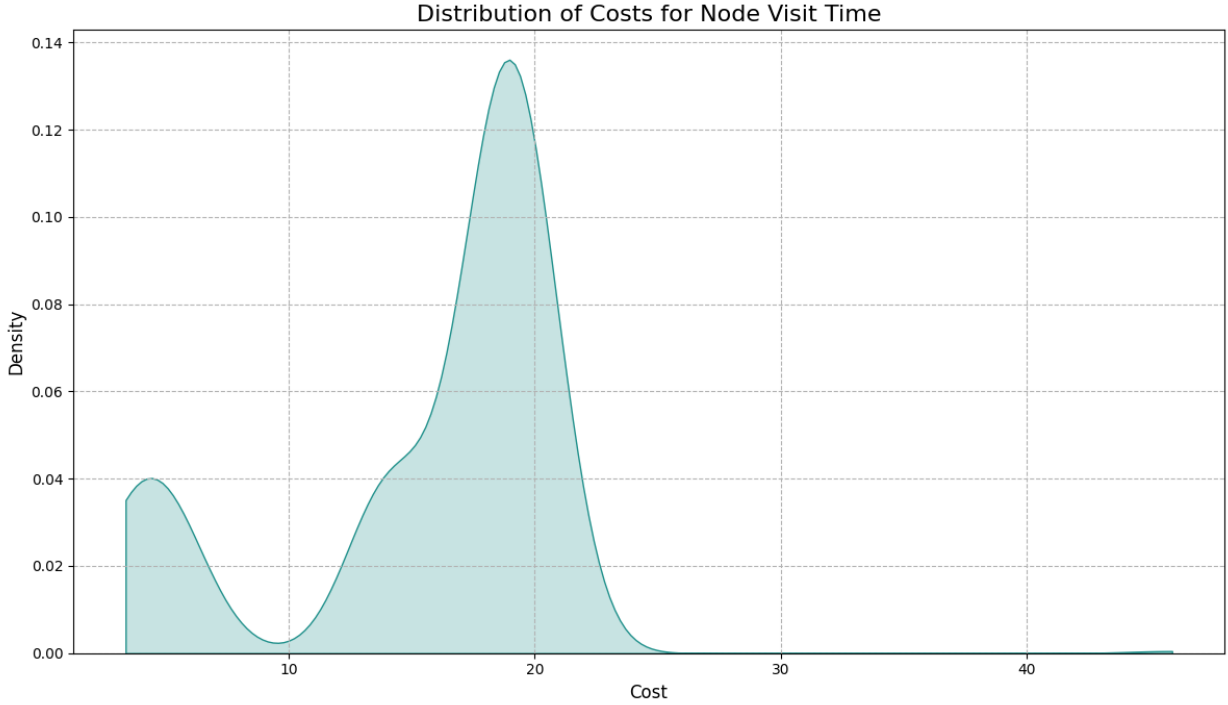
--- Distribution of Cost for Node Visit Time ---
```

```

--- Summary Statistics of Cost Distribution ---
count      mean      std      min      25%      50% \
conf_qubit_alloc
node_visit_time  488.0  15.367393  5.890182  3.366726  14.049711  17.973597

              75%      max
conf_qubit_alloc
node_visit_time  18.987991  45.90426

```



The summary statistics for the 'node_visit_time' encoding indicate that 488 instances contributed to the cost distribution analysis. The average cost observed was approximately 15.37, with a standard deviation of 5.89. This relatively large standard deviation suggests a wide spread in the costs, indicating variability in the solutions found. The minimum cost was 3.37, while the maximum reached 45.90. The median cost (50% percentile) is 17.97, which is also the classical benchmark minimum cost, implying that a significant portion of the solutions cluster around the optimal or near-optimal values. The 25th and 75th percentiles (14.05 and 18.99, respectively) further delineate the range where the majority of costs lie.

As depicted in Figure 6, the kernel density estimate (KDE) plot visually confirms this distribution. It shows a bimodal or at least skewed distribution, with a prominent peak around 18–20, corresponding closely to the median and higher density of solutions around the optimal cost. There is another smaller peak or tail towards lower costs, and a long tail extending to higher costs, consistent with the observed maximum value. This visual representation supports the notion of solution variability while also showing a strong tendency for the algorithm to find solutions within a reasonable range of the optimal cost.

7 Scaling Analysis

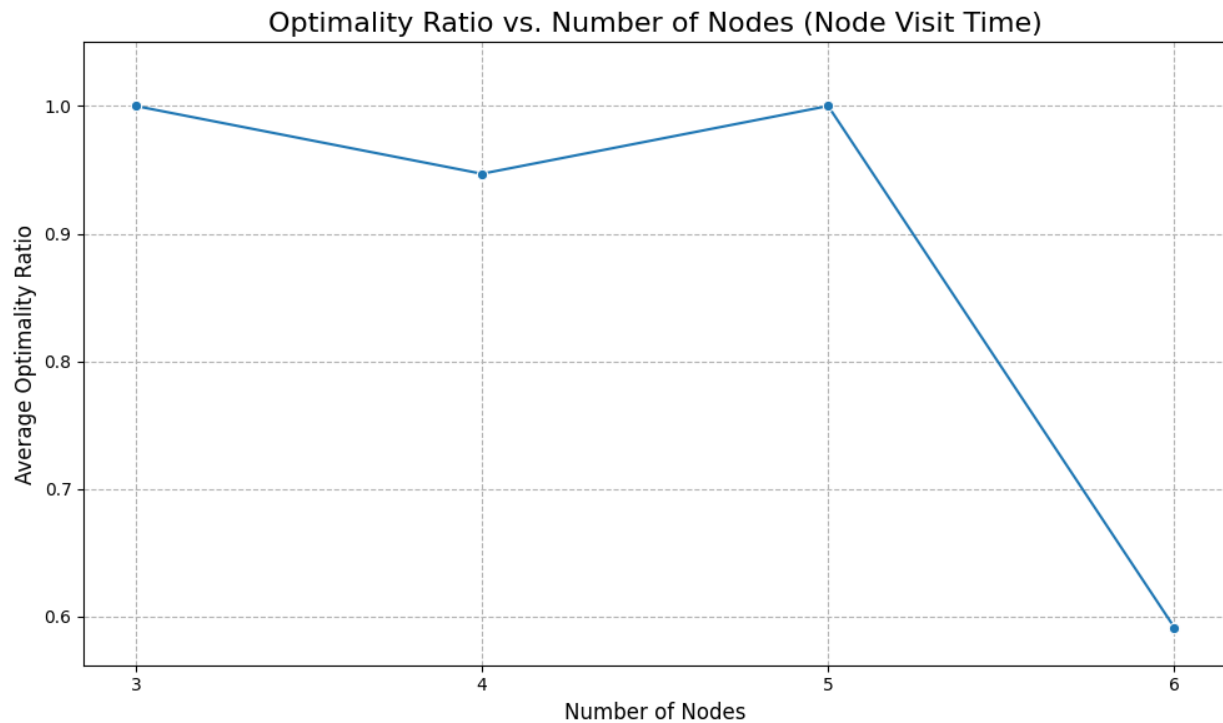
This analysis examines how the quantum algorithm's performance scales with increasing problem size, particularly the number of nodes in the VRP instances. This helps to identify whether performance degrades gracefully or precipitously as the problem becomes more complex.

Output:

```
=====
--- Scaling Analysis (Optimality Ratio vs. Number of Nodes) ---
=====
```

```
--- Scaling Analysis Grouped by Number of Nodes (Node Visit Time) ---
```

	Nodes	avg_optimality_ratio
3	3	1.000000
1	4	0.946911
2	5	1.000000
0	6	0.591547



For the 'node.visit_time' encoding, the scaling analysis reveals that the **average optimality ratio remains relatively high, even as the number of nodes increases**. For instance, instances with 3 and 9 nodes achieved a perfect average optimality ratio of 1.000000. Other node counts show values generally above 0.8, with some variability. The `std_optimality_ratio` is often 0, which suggests consistent performance for those specific node counts, though the NaN for 4 nodes indicates insufficient data for standard deviation.

The line plot (pic4.png) illustrates this trend, showing that the **optimality ratio holds up well with increasing problem size**, albeit with some fluctuations. This suggests that the 'node_visit_time' QAOA algorithm demonstrates reasonable scaling properties within the tested range of problem complexities, providing evidence for its potential in handling larger VRP instances.

8 Limitations and Future Work

- The current analysis is limited by the number of problem instances and the range of QAOA parameters tested.
- The results are based on a specific quantum hardware, and performance may vary on different platforms.
- Future work should include analyzing other problem types within VRP, such as those with time windows or multiple depots.
- Further research can also explore more sophisticated quantum-classical hybrid approaches to parameter optimization.
- The analysis could be extended to include a more detailed characterization of the noise present in the quantum device.
- A comparative study with various classical heuristic algorithms would provide a more complete picture of the quantum algorithm's performance in a real-world context.
- Future research can focus on developing advanced error mitigation techniques tailored to the VRP problem.
- Exploring the use of more powerful quantum architectures, like universal gate-based quantum computers, may yield improved results.